

Particle Detector report

Callum Williamson

May 14, 2026

Abstract

An object oriented C++ program was developed to simulate a particle detector. Abstract base classes, inheritance and polymorphism were employed to model particle and sub-detector hierarchies. Composition and dynamic memory were implemented to store sub-detector objects within a detector system class, allowing for particles to be passed through multiple sub-detectors. The program demonstrated particle dependent detector responses, as shown, muons were detected by trackers and muon-chambers but not by calorimeters.

1 Introduction

This project develops an object oriented C++ program to simulate a particle detector. The detector was composed of sub-detectors, like the example of the CMS detector in Figure 1. Different particle types are passed through the detector, with the sub-detectors response depending on the physical properties of the particle. My code modelled this based on the responses indicated in Figure 1 and outlined in [1].

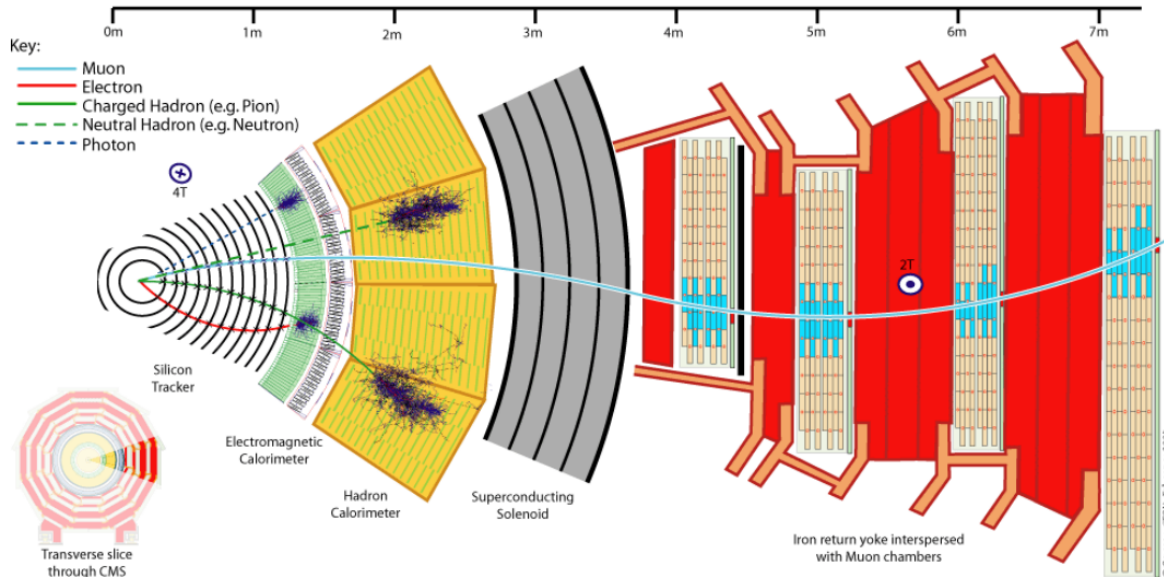


Figure 1: Cross sectional schematic of the CMS detector [2]. Sub-detectors of note are the: silicon tracker, muon chamber, electromagnetic and hadron calorimeters. The particles interaction is illustrated by their paths which can be distinguished using the key.

The main aim of the project was to practice object oriented programming principles, building a clear

and extensible class structure for both particles and detectors. The program possesses separate class hierarchies for both particles and sub-detectors, with base classes `Particle` and `SubDetector`, respectively. Inheritance and polymorphism simplify the code and reduce repetition for the derived objects. `SubDetector` objects are owned via unique pointers by a large detector class, `DetectorSystem`, which acts as a container class.

This report describes the key choices made throughout the design process, first considering the decisions required to meet the minimum functionality to a high standard, before discussing the choices that enabled more advanced features. The final sections discuss how the program is used in `main()`, where it is successful, and how it could be improved.

2 Design process & considerations

As mentioned previously, the key design considerations are discussed in two parts: minimum functionality and advanced features. These are further divided around the three main components of the program: the `Particle` and `SubDetector` hierarchies and the `DetectorSystem` class.

2.1 Minimum functionality

Table 1: Summary of where the minimum functionality requirements were implemented.

Requirement	Implementation
1. Abstract base classes for a particle and sub-detector.	<code>Particle</code> and <code>SubDetector</code> .
2. Derived classes for sub-detectors: trackers, muon chambers and calorimeters.	<code>Tracker</code> , <code>MuonChamber</code> , <code>EMCalorimeter</code> and <code>HadronCalorimeter</code> .
3. Derived classes for particles: electrons, muons, photons and hadrons.	<code>Electron</code> , <code>Muon</code> , <code>Photon</code> and derived <code>Hadron</code> objects.
4. Virtual functions to set and get characteristics of the detectors and particles.	Not employed, just normal setters and getters.
5. Four-momentum is stored as a class in each particle.	<code>FourMomentum</code> composed within <code>Particle</code> .
6. A detector that returns the energy of the particle it detected.	<code>detect()</code> .
7. A generic detector class that stores sub-detectors.	<code>DetectorSystem</code> stores <code>SubDetector</code> using <code>std::vector<std::unique_ptr<SubDetector>></code> .

Although requirement 4 of Table 1 refers to virtual setters and getters, these were kept non-virtual within all hierarchies as their behaviour is identical across derived classes, avoiding unnecessary over-riding.

2.1.1 Particles

`Particle` was created as an abstract base class from which the particles are derived, meeting requirement 1 of Table 1. Figure 2 shows the `Particle` base class data members, which were chosen because they represent intrinsic properties shared by all particles. Figure 2 also shows the composition relationship between `FourMomentum` and `Particle`, satisfying requirement 5 in Table 1. This is implemented using a `std::unique_ptr`, enforcing exclusive ownership of `four_momentum`. The storage of `FourMomentum` components was changed from Assignments 4's raw pointers to `std::unique_ptr` to increase memory safety. The decision to implement dynamic memory meant that custom move/copy semantics were required. The validation of `FourMomentum` simply ensures the energy is not negative using `validate_E`, as seen in Figure 2.

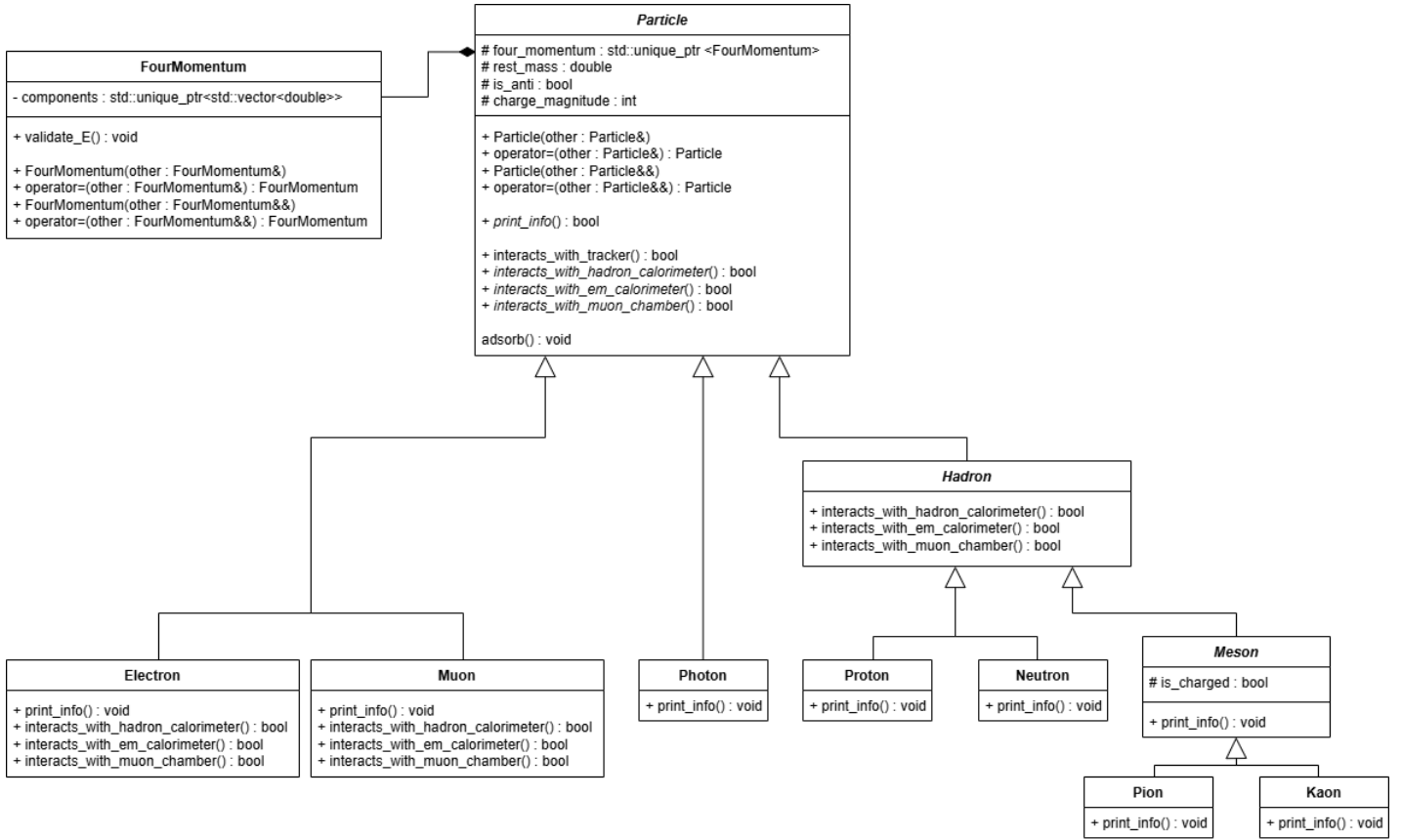


Figure 2: UML class diagram for the **Particle** hierarchy. Note no default, parametrised constructors or destructors are included to maintain simplicity.

Interaction logic was chosen to be stored in the **Particle** hierarchy rather than the **SubDetector** hierarchy. This allowed for shared behaviour, such as charged particles interacting with the **Tracker** in Code snippet 1, to be inherited by derived particles, reducing repetition and improving extensibility. New particles can reuse existing logic where appropriate, while any particle specific interactions can be distinctly implemented. This follows the open closed principle, whereas if we chose to store in the **SubDetector** hierarchy, every derived **SubDetector** would have to be updated when a new particle is created. Detector specific conditions, such as **status**, remain within the **SubDetector** classes, separating whether interaction is physically possible from whether detection actually occurs, increasing flexibility and clarity.

```

1 bool Particle::interacts_with_tracker() const
2 {
3     if (charge_mag == 0)
4     {
5         return false; // Neutral Particles don't interact with Trackers.
6     }
7     else
8     {
9         return true; // Charged Particles interact with Trackers.
10    }
11 }

```

Code snippet 1: **Particle** member function **interacts_with_tracker**. As seen in Figure 2, it is the only **interacts_with** member function defined in **Particle** and not overridden in the derived classes.

As seen in Figure 2, the member functions, **interacts_with**, were used instead of storing **SubDetector**

names in a `vector` of `std::string` or `enum` data types. This decision allows for the logic behind the interaction rule to be expressed where defined, as shown by Code snippet 1, rather than stored only as a fixed result. Hence it also improves adaptability, since any future conditions such as energy thresholds can be added inside the relevant member function. On top of this, it will be seen that the design choice greatly simplifies the detection, with `SubDetector` only having to call a single `bool` member function.

The class hierarchy seen in Figure 2 was decided upon based on shared properties of the particles required in Table 1. `Photon` was derived directly from `Particle` due to it possessing distinct properties and not sharing behaviour with the other particles. It became clear that the inherited data members, `rest_mass` and `is_anti`, were redundant for `Photon` as it has no mass and is its own anti-particle. However, since they are not damaging and a necessity for most of the particles, it was worthwhile for them to remain as base data members. To mitigate this the polymorphic `print_info()` addresses the fact that photons are their own anti-particles. The same was done for the neutral `Pion`.

The charged leptons, `Electron` and `Muon`, were not grouped under an intermediate base class due to interaction behaviour not being shared. Hadronic particles were grouped under an intermediate base class, where the common interaction logic is implemented in `Hadron` and consequently inherited by the derived objects. A further intermediate base class, `Meson`, was introduced because mesons follow a general construction logic depending on the distinct data member `is_charged`. As shown in Code snippet 2, this parameter controls if a charged or neutral `Meson` is instantiated, setting the appropriate `charged_mag` and `rest_mass`. For charged mesons, `is_anti` then determines the charge sign, while the polymorphic `print_info` functions distinguish neutral particles that are their own anti-particles, such as `Pion`, from mesons with distinct anti-particles, such as `Kaon`. This reduces repeated constructor and output logic across derived `Meson` objects. Baryons were not given an intermediate base class because their construction logic is not consistent, however the hierarchy still allows a baryon base class to be introduced later if more shared behaviour is added.

```

1 Meson::Meson(double E, double px, double py, double pz, bool is_anti,
2             bool is_charged, double neutral_rest_mass, double charged_rest_mass)
3     // Calling the intermediatebase class parameterised constructor.
4     : Hadron(E, px, py, pz, is_charged ? charged_rest_mass : neutral_rest_mass
5             , is_anti, is_charged ? 1 : 0),
6     is_charged(is_charged)
7 {
8     if (Debug::show_messages)
9     {
10         std::cout << "Calling Meson Parameterised Constructor." << std::endl;
11     }
12 }

```

Code snippet 2: `Meson` parameterised constructor. Arguments `neutral_rest_mass` and `charged_rest_mass` are fixed by the derived `Particle`, `is_charged` is chosen by the user and determines what type of `Meson` is constructed. The charge of the charged mesons is determined by `is_anti`, with the anti-particle arbitrarily set as the negative particle.

2.1.2 SubDetectors

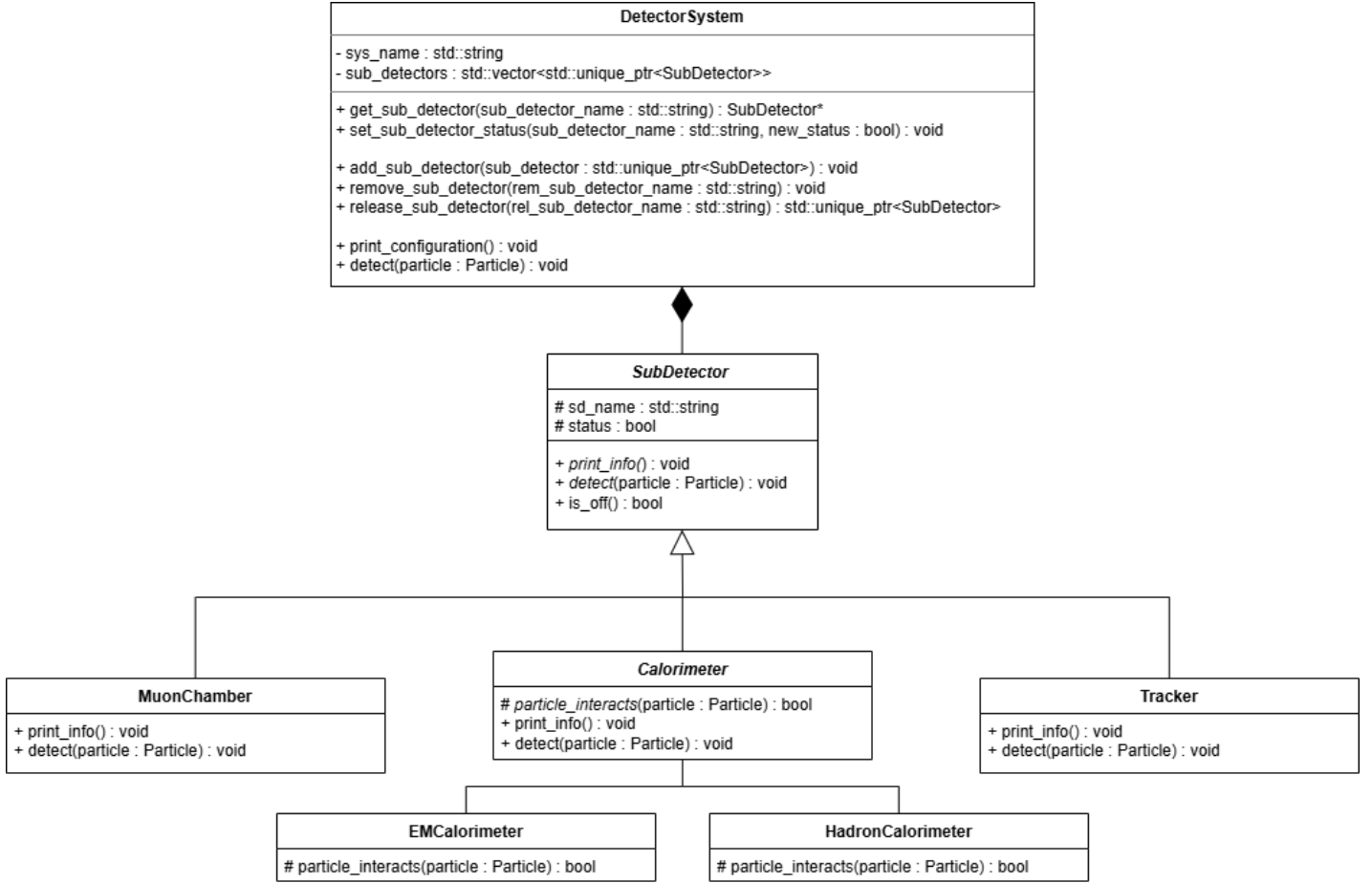


Figure 3: UML class diagram for the **SubDetector** hierarchy and its composition in **DetectorSystem**. Note, no constructors or destructors are included to maintain simplicity.

The **SubDetector** hierarchy was initially developed using an abstract base class and one derived class, allowing the core interface to be tested before adding more derived classes. Figure 3 shows that the base class has two data members `sd_name` and `status`. `sd_name` simply distinguishes between **SubDetector** objects while `status` controls whether a **SubDetector** is active. Since **SubDetector** manages no dynamically allocated data members, the default compiler copy/move operations are sufficient, hence no custom copy/move semantics were required.

To meet requirement 6 of Table 1, **Particle** objects must be passed through the derived **SubDetector**, which returns the particle's energy. This was implemented via a virtual base member function `detect`. As virtual it allowed each derived **SubDetector** to define its own detection response while sharing a common interface. This common interface was utilised in **DetectorSystem** to pass a **Particle** through all contained **SubDetector** objects clearly and efficiently, as discussed later.

Following the interaction design discussed in Section 2.1.1, the `SubDetector::detect` uses the passed **Particle**'s relevant `interacts_with` function to determine whether detection is physically possible. Code snippet 3, line 9, demonstrates the simplicity of this approach. The omission of `const` in the `particle` argument is discussed later in the advanced features section.

```

1 void MuonChamber::detect(Particle& particle) const
2 {
3     // Check status of SubDetector.
4     if (is_off())
5     {
6         return; // Exit function.
7     }
8
9     if (particle.interacts_with_muonchamber())
10    {
11        std::cout << "Muon Chamber (" << sd_name << ") measures the energy of
12        the particle as: " << particle.get_four_momentum().get_E() << "
13        MeV" << std::endl;
14        std::cout << "The particle's true energy is: " << particle.
15        get_four_momentum().get_E() << " MeV" << std::endl;
16        std::cout << std::endl; // Line break.
17    }
18    else
19    {
20        std::cout << "Particle does not interact with the muon chamber (" <<
21        sd_name << "), no energy measurement." << std::endl;
22        std::cout << std::endl; // Line break.
23    }
24 }

```

Code snippet 3: Derived `MuonChamber::detect` definition acts as an example for how the `Particle` object's `interacts_with` function is utilised in line 9.

2.1.3 DetectorSystems

`DetectorSystem` satisfies the final requirement of Table 1, by acting as a generic detector class that stores `SubDetector` objects. As presented in Figure 3, the `sys_name` data member differentiates `DetectorSystem` objects. The other data member, as declared in Code snippet 4, is a `vector` of unique pointers owning `SubDetector` objects. This is what gives `DetectorSystem` its utility of being a container of multiple derived `SubDetector` objects polymorphically.

```

1 private:
2     std::vector<std::unique_ptr<SubDetector>> sub_detectors;

```

Code snippet 4: Data member for `DetectorSystem` to manage the ownership of multiple `SubDetector` classes.

`std::unique_ptr` was chosen over `std::shared_ptr` because each `SubDetector` should belong to a single `DetectorSystem`, reflecting exclusive ownership. Shared ownership would add unnecessary complexity, as the same physical `SubDetector` component would not normally be part of multiple `DetectorSystem` objects. Copy constructors and assignment operators were deleted, to prevent copying of unique pointers. Duplicating a whole `DetectorSystem` would also not represent a meaningful physical operation. Move construction and assignment were defaulted as `std::unique_ptr` already provides safe ownership transfer.

```

1 void add_sub_detector(std::unique_ptr<SubDetector> sub_detector);

```

Code snippet 5: `DetectorSystem` member function interface to add `SubDetector` objects to the container.

`SubDetector` objects are added to the `DetectorSystem` using the member function stated in Code snippet 5. The full `DetectorSystem` configuration can be displayed using the member function `print_configuration`, which iterates through the stored `SubDetector` objects in the same manner as

`DetectorSystem::detect` in Code snippet 6. As discussed in Section 2.1.2, `SubDetector::detect` is virtual, so each stored `SubDetector` can be called via the same interface, as seen in line 8.

```

1 void DetectorSystem::detect(Particle& particle) const
2 {
3     std::cout << "--- Passing particle through DetectorSystem (" << sys_name
4         << ") ---" << std::endl;
5     std::cout << std::endl; // Line break.
6
7     for (const auto& sub_detector : sub_detectors)
8     {
9         sub_detector->detect(particle);
10    }
11
12    std::cout << "-- Finished passing particle through DetectorSystem (" <<
13        sys_name << ") ---" << std::endl;
14    std::cout << std::endl; // Line break.
15 }

```

Code snippet 6: `DetectorSystem` member function implementation to pass the `Particle` through the all the owned `SubDetector` objects.

2.2 Advanced features

The advanced features section includes code that implements higher level C++ techniques or goes beyond the minimum functionality.

The advanced C++ feature `namespace`, was employed throughout the whole program to globally control whether debug messages are printed or not, as shown in Code snippet 7. These messages are useful for debugging, however overwhelming for a general user, hence the utility to switch on or off is valuable. The choice to use a `namespace` instead of a simple global variable `show_messages` can avoid name clashes between other variables similarly named. This organisational benefit is clearly demonstrated by the `DetectorSystem` specific messages, as shown in Code snippet 8.

```

1 namespace Debug
2 {
3     inline bool show_messages = false;
4 }

```

Code snippet 7: `Debug` namespace to control globally whether debug messages, such as “Calling Particle Destructor.” are printed.

```

1 if (DSMessages::show_messages)
2 {
3     std::cout << "Added SubDetector (" << new_sub_detector->get_sd_name() << "
4         << ") to DetectorSystem (" << sys_name << ")." << std::endl;
5     std::cout << std::endl;
6 }

```

Code snippet 8: Demonstration of `DetectorSystem` specific messages controlled by the `namespace DSMessages`. This snippet is taken from the `add_sub_detector` member function, whose interface is shown in Code snippet 5.

2.2.1 SubDetectors

The data member `status` goes beyond the minimum functionality, with switching a `SubDetector` on or off adding physical realism. As will be seen in Section 2.2.2, a `SubDetector` contained within a `DetectorSystem` can also have its `status` modified by the `DetectorSystem` container.

The `Particle` argument in `SubDetector::detect` was not `const` to support the additional feature of `Calorimeter` objects stopping the detected particles completely [1], hence altering the `Particle` object's `four_momentum`. This was implemented using the `Particle` base member function `absorb()` inside `Calorimeter::detect` function, as shown by Code snippets 9 and 10. A friend function approach was considered as I initially thought it would allow a more physically aligned process to be programmed, with the `SubDetector` altering the `four_momentum`. However, this wasn't the case and hence granting the `SubDetector` access to the `Particle` protected data members produced no value. Instead `Particle::absorb` was simpler and preserved encapsulation while still being applied within the `SubDetector::detect`.

```

1 void Particle::absorb()
2 {
3     four_momentum->set_E(rest_mass);
4     four_momentum->set_px(0.0);
5     four_momentum->set_py(0.0);
6     four_momentum->set_pz(0.0);
7 }
8 }

```

Code snippet 9: `Particle::absorb` alters the `Particle`'s `four_momentum`, returning its energy as its `rest_mass` and hence it is stationary. Note, all zero for `Photon` and therefore acts as if it is fully absorbed.

```

1 void Calorimeter::detect(Particle& particle) const
2 {
3     // Check status of SubDetector.
4     if (is_off())
5     {
6         return; // Exit function.
7     }
8
9     if (particle_interacts(particle)) // Removes code repetition by having
10    detect differ in the derived classes.
11    {
12        std::cout << "Calorimeter (" << sd_name << ") measures the energy of
13        the particle as: " << particle.get_four_momentum().get_E() << "
14        MeV" << std::endl;
15        std::cout << "The particle's true energy is: " << particle.
16        get_four_momentum().get_E() << " MeV" << std::endl;
17        std::cout << "The particle is absorbed." << std::endl;
18        particle.absorb();
19        std::cout << std::endl; // Line break.
20    }
21    else
22    {
23        std::cout << "Particle does not interact with the calorimeter (" <<
24        sd_name << "), no energy measurement." << std::endl;
25        std::cout << std::endl; // Line break.
26    }
27 }
28 }

```

Code snippet 10: Demonstration of how `Calorimeter` objects apply the `Particle::absorb` function to mimic the reality that the particles are completely stopped.

2.2.2 DetectorSystems

Member functions for both removing and releasing `SubDetector` objects were added to the `DetectorSystem` container to add higher functionality. The `remove_sub_detector` function removes and destroys the

owned `SubDetector`, as seen in Code snippet 11. This was physically motivated by the possibility of a detector component failing and needing to be removed or replaced. `release_sub_detector` transfers ownership from the container to an external `std::unique_ptr`. This allows the user to extract the `SubDetector` from the container while preserving the object itself, for example if the component is useful alone or in another `DetectorSystem`. It is applied in much the same way as Code snippet 11.

```

1 void DetectorSystem::remove_sub_detector(const std::string&
   rem_sub_detector_name)
2 {
3     // Find which index the SubDetector is at in the vector, if it exists.
4     std::vector<std::unique_ptr<SubDetector>>::iterator it = std::find_if(
5         sub_detectors.begin(),
6         sub_detectors.end(),
7         [&rem_sub_detector_name](const std::unique_ptr<SubDetector>&
           sub_detector)
8         {
9             return sub_detector->get_sd_name() == rem_sub_detector_name;
10        }
11    );
12
13    if (it == sub_detectors.end())
14    {
15        std::cerr << "Error: SubDetector (" << rem_sub_detector_name << ") not
           found in DetectorSystem (" << sys_name << ")." << std::endl;
16        std::cout << std::endl;
17        return;
18    }
19
20    sub_detectors.erase(it);
21    if (DSMessages::show_messages)
22    {
23        std::cout << "Removed SubDetector (" << rem_sub_detector_name << ")
           from DetectorSystem (" << sys_name << ")." << std::endl;
24        std::cout << std::endl;
25    }
26 }

```

Code snippet 11: `DetectorSystem` member function to remove a `SubDetector` it owns. A lambda function is used in lines 7 to 10.

Both member functions utilise a lambda function, an advanced C++ technique, when finding if the selected `SubDetector` is contained within the `DetectorSystem`. This is carried out using the `algorithm` `std::find_if` function, which searches through the containing data member `sub_detectors`. A lambda function returns `true` if the `SubDetector` name of that iteration matches the requested name. `std::find_if` then returns the iterator associated with the matching `SubDetector` or `sub_detectors.end()` if no match is found. The lambda function maintains clarity of the process because the search condition is defined directly where it is needed. Defining the same function elsewhere would achieve the same result but make this simple procedure less visible. The lambda function therefore acts as a beneficial, small local attachment to `std::find_if`, as seen in in lines 7 to 10 in Code snippet 11. Note the member function `get_sub_detector`, mentioned below and in Code snippet 12, uses the same `std::find_if` and lambda function logic.

A wrapper member function `set_sub_detector_status`, as seen in Code snippet 12, was created to modify the `status` of a `SubDetector` contained in a `DetectorSystem`, increasing the functionality. It does so by using `get_sub_detector` to return a raw non-owning pointer to a contained `SubDetector`, if the requested name exists. It is important that this pointer does not transfer ownership, so that the `DetectorSystem` object's `std::unique_ptr` remains intact. The returned pointer is only used to access and modify the contained `SubDetector`.

```

1 void DetectorSystem::set_sub_detector_status(const std::string&
   sub_detector_name, bool new_status)
2 {
3     SubDetector* sub_detector = get_sub_detector(sub_detector_name);
4
5     if (!sub_detector)
6     {
7         std::cerr << "Error: SubDetector (" << sub_detector_name << ") not
           found in DetectorSystem (" << sys_name << ")." << std::endl;
8         std::cout << std::endl;
9         return;
10    }
11
12    sub_detector->set_status(new_status);
13    if (DSMessages::show_messages)
14    {
15        std::cout << "Set SubDetector (" << sub_detector_name << ") status to
           -" << (new_status ? "on" : "off")
16        << "- in DetectorSystem (" << sys_name << ")." << std::endl;
17        std::cout << std::endl;
18    }
19 }

```

Code snippet 12: `DetectorSystem` member function to set a contained `SubDetector` object's status. It employs another member function `get_sub_detector` to return a raw non-owning pointer of `SubDetector`, as seen in line 3.

3 Results

The following results show how the developed classes are used in the main program. The outputs are presented in two parts: minimum functionality and advanced features, analogous to the previous section. Screenshots of the terminal outputs are used to show the program behaviour.

3.1 Minimum functionality

The minimum functionality was tested by constructing a `DetectorSystem` named `det_sys_CMS`, which is composed of the major CMS `SubDetector` components seen in Figure 1. The resulting configuration was printed using `print_configuration` to verify that each `SubDetector` had been successfully added to the container.

```

--- Passing particle through DetectorSystem (CMS Detector System) ---

Tracker (Silicon Tracker) measures the energy of the particle as: 130 MeV
The particle's true energy is: 130 MeV

Particle does not interact with the calorimeter (Crystal Electromagnetic Calorimeter), no energy measurement.

Particle does not interact with the calorimeter (Hadron Calorimeter), no energy measurement.

Muon Chamber (Muon Chamber) measures the energy of the particle as: 130 MeV
The particle's true energy is: 130 MeV

--- Finished passing particle through DetectorSystem (CMS Detector System) ---

```

Figure 4: Screenshot of the terminal's output for the `det_sys_CMS::detect` of the Muon object.

A `Muon` object is created and passed through `det_sys_CMS`, with the expectation that it will be detected by the `Tracker` and `MuonChamber` but not by the `Calorimeter` objects as they deposit little energy. The success of this can be seen in the `DetectorSystem::detect` output in Figure 4. For each successful detection, the detected energy and its true energy are outputted. These values are equal because the detector response is modelled as perfect.

3.2 Advanced features

The first advanced functionality tested was the wrapper member function `DetectorSystem::set_detector_status`. This was tested by switching off the `MuonChamber` before passing the same `Muon` through the `DetectorSystem` again. The output can be seen in Figure 5 with, as expected, no detection occurring for the `MuonChamber` as it is inactive. This demonstrates that the wrapper function successfully modifies the `SubDetector`'s `status` via `DetectorSystem`.

```
Demonstrating the use of a wrapper function to alter the status of a contained SubDetector.
Demonstrating how this alters the detection process.
Set SubDetector (Muon Chamber) status to -off- in DetectorSystem (CMS Detector System).

--- Passing particle through DetectorSystem (CMS Detector System) ---

Tracker (Silicon Tracker) measures the energy of the particle as: 130 MeV
The particle's true energy is: 130 MeV

Particle does not interact with the calorimeter (Crystal Electromagnetic Calorimeter), no energy measurement.
Particle does not interact with the calorimeter (Hadron Calorimeter), no energy measurement.

SubDetector (Muon Chamber) is off. No detection.

--- Finished passing particle through DetectorSystem (CMS Detector System) ---
```

Figure 5: Screenshot of the terminal's output for the `det_sys_CMS::detect` of the `Muon` object after the wrapper function `det_sys_CMS.set_sub_detector_status("Muon Chamber", false)` has been applied.

The next advanced functionality demonstrated was the `Calorimeter::detect` and how it affects `Particle` objects that interact with it. Like the `Tracker` and `MuonChamber`, the `Calorimeter` objects print both the measured and true energy. It then applies the `absorb` member function with this behaviour being shown in Figure 6 by comparing the `print.info` output before and after detection.



```
--- Pion properties ---
Four momenta: (500, 4, 5, 6) MeV
Rest mass: 134.98 MeV
Is an anti-particle (true/false): false
Charge: 0e (Neutral Meson)
Note: Neutral Pions are their own anti-particles.

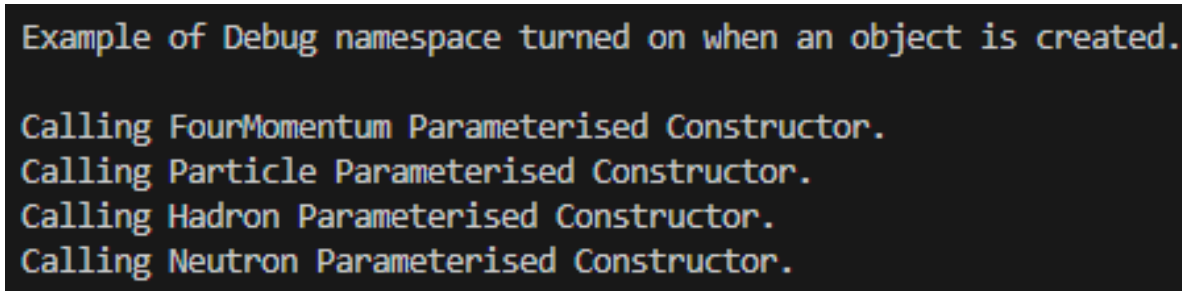
--- Pion properties ---
Four momenta: (134.98, 0, 0, 0) MeV
Rest mass: 134.98 MeV
Is an anti-particle (true/false): false
Charge: 0e (Neutral Meson)
Note: Neutral Pions are their own anti-particles.
```

Figure 6: Screenshot of the terminal's output for before and after a `Pion` has been detected by a `HadronCalorimeter`. Appropriate change in `Pion`'s `four_momentum` is illustrated.

Another `DetectorSystem`, `det_sys_2`, is used to test the ownership management functions `remove_sub_detector` and `release_sub_detector`. The output shows that a `SubDetector` can either be removed from the container and destroyed or released and used independently.

The utility of `namespaces` is shown in Figure 7, by turning `Debug::show_messages` to `true` and creating an object. It can be seen that these messages are useful for debugging, however they substantially

bloat the main and hence the ability for them to be turned off is essential for the general user. Similar can be done for `DSMessages`.

A screenshot of a terminal window with a black background and yellow text. The text displays a sequence of debug messages triggered by the creation of a Neutron object. The messages are: 'Example of Debug namespace turned on when an object is created.', 'Calling FourMomentum Parameterised Constructor.', 'Calling Particle Parameterised Constructor.', 'Calling Hadron Parameterised Constructor.', and 'Calling Neutron Parameterised Constructor.'

```
Example of Debug namespace turned on when an object is created.  
Calling FourMomentum Parameterised Constructor.  
Calling Particle Parameterised Constructor.  
Calling Hadron Parameterised Constructor.  
Calling Neutron Parameterised Constructor.
```

Figure 7: Screenshot of the terminal’s output for when a `Neutron` is created while `Debug` messages are on.

4 Discussion and conclusions

The main program successfully demonstrates an object oriented particle detector simulation. The abstract base classes, `Particle` and `SubDetector`, provide common interfaces, while inheritance and polymorphism allow different derived `Particle` objects to pass through a `DetectorSystem` that owns derived `SubDetector` objects. This also made the design extensible, with the addition of threshold energies to the `interacts_with` functions serving as a simple example.

The advanced features increased physical realism and usability of the program. For example, absorption showed how `SubDetector` interactions can modify a `Particle`’s state. Future improvements would be to include detector imperfections, possibly introducing fluctuations with a Gaussian distribution. The current `FourMomentum` validation could also be improved by rejecting unphysical particles that are inconsistent with $E^2 = p^2 + m^2$.

References

- [1] CMS Collaboration. *The CMS detector*. URL: <https://cms.cern/detector> (visited on 09/05/2026).
- [2] Steven Maria A Mortier. “Exploring top-quark and Z boson coupling in top pair production in association with a Z boson”. PhD thesis. Gent U., 2020.